

Python Exception Handling

A Textbook, Workbook, Interactive Guide, and Professional Reference

Build programs that fail safely

Learn to predict errors, read tracebacks, recover gracefully, clean up resources, raise meaningful exceptions, and write production-quality Python.

Beginner → Intermediate → Advanced → Professional

Built from the uploaded exception-handling notebooks, rewritten and reorganized into a cohesive curriculum.

How to Use This Book

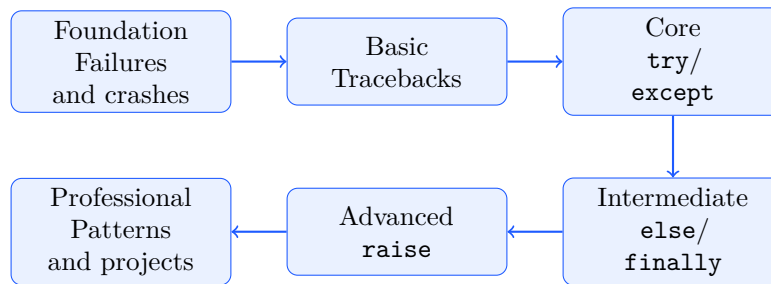
This resource is designed for a learner with zero prior knowledge of exception handling. Read each module in order, type the examples yourself, predict the output before running code, and complete the workbook tasks without looking for Python solutions. Every topic follows this learning path: Introduction → Intuition → Formal Syntax → Execution Flow → Examples → Practice → Professional Use.

Learning Objectives

By the end, you will be able to:

- Explain what exceptions are and why professional programs need them.
- Read Python tracebacks from bottom to top and locate the failing line.
- Use `try`, `except`, `else`, and `finally` correctly.
- Catch specific exception types without hiding programming bugs.
- Raise exceptions intentionally to enforce business rules.
- Re-raise exceptions after logging or adding context.
- Design robust input, file, dictionary, list, network, and validation workflows.

Learning Roadmap



Progress Tracker

Module	Skill	Done
1	Explain exceptions and failure prediction	☐
2	Read and debug tracebacks	☐
3	Use specific <code>try/except</code> blocks	☐
4	Use <code>else</code> and <code>finally</code> for clean workflows	☐
5	Raise and re-raise exceptions professionally	☐
6	Complete projects and interview preparation	☐

Contents

1	Foundations: Why Exceptions Exist	1
1.1	Introduction	1
1.2	The Problem Exceptions Solve	1
1.3	Formal Explanation	1
1.4	Visual Representation	2
1.5	Syntax Preview	2
1.6	Step-by-Step Execution	2
1.7	Five Examples	3
1.8	Real-World Applications	4
1.9	Edge Cases	4
1.10	Common Mistakes	4
1.11	Workbook Tasks	5
1.12	Interview Questions	5
2	Tracebacks: Reading Python’s Failure Report	6
2.1	Introduction	6
2.2	Formal Explanation	6
2.3	Traceback Anatomy	6
2.4	Visual Debugging Process	7
2.5	Five Examples	7
2.6	Applications, Edge Cases, and Mistakes	8
2.7	Practice Tasks with Pseudocode	8
2.8	Interview Questions	9
3	Core Handling: try and except	10
3.1	Introduction	10
3.2	Intuition First	10
3.3	Formal Syntax	10
3.4	Decision Tree	11
3.5	Five Examples	11
3.6	Comparison Table	12
3.7	Real-World Applications	13
3.8	Edge Cases	13
3.9	Common Mistakes	13
3.10	Practice Tasks with Pseudocode	13
3.11	Interview Questions	14
4	Clean Control Flow: else and finally	15
4.1	Introduction	15
4.2	Intuition First	15
4.3	Formal Syntax	15
4.4	Execution Flow	15

4.5	Five Examples	16
4.6	Real-World Applications	17
4.7	Edge Cases	18
4.8	Common Mistakes	18
4.9	Practice Tasks with Pseudocode	18
4.10	Interview Questions	18
5	Raising Exceptions Intentionally	19
5.1	Introduction	19
5.2	Intuition First	19
5.3	Formal Syntax	19
5.4	Raise-and-Catch Lifecycle	19
5.5	Five Examples	20
5.6	Real-World Applications	21
5.7	Edge Cases	21
5.8	Common Mistakes	22
5.9	Practice Tasks with Pseudocode	22
5.10	Interview Questions	22
6	Professional Patterns and Production Thinking	23
6.1	Best-Practice Mind Map	23
6.2	Professional Design Principles	23
6.3	Use Cases vs Anti-Patterns	23
6.4	Challenge Section	23
6.5	Capstone Practice Tasks with Pseudocode	24
6.6	Professional Interview Set	25
7	Cheat Sheets, Revision Notes, Glossary, and Index	26
7.1	Quick Reference Guide	26
7.2	Exception Selection Card	26
7.3	Chapter Summaries	26
7.4	Final Revision Questions	27
7.5	Glossary	27
7.6	Further Learning Recommendations	27

Module 1

Foundations: Why Exceptions Exist

1.1 Introduction

An **exception** is an event that interrupts the normal order of a program. It usually means, “Python reached a situation it cannot continue from safely.” Exception handling is the skill of preparing a safe response instead of letting the program crash.

Intuition First

Exceptions are like emergency exits in a building. The building is still designed for normal movement through hallways, but if smoke appears, people need a separate path that protects them. Your program needs the same kind of emergency route.

1.2 The Problem Exceptions Solve

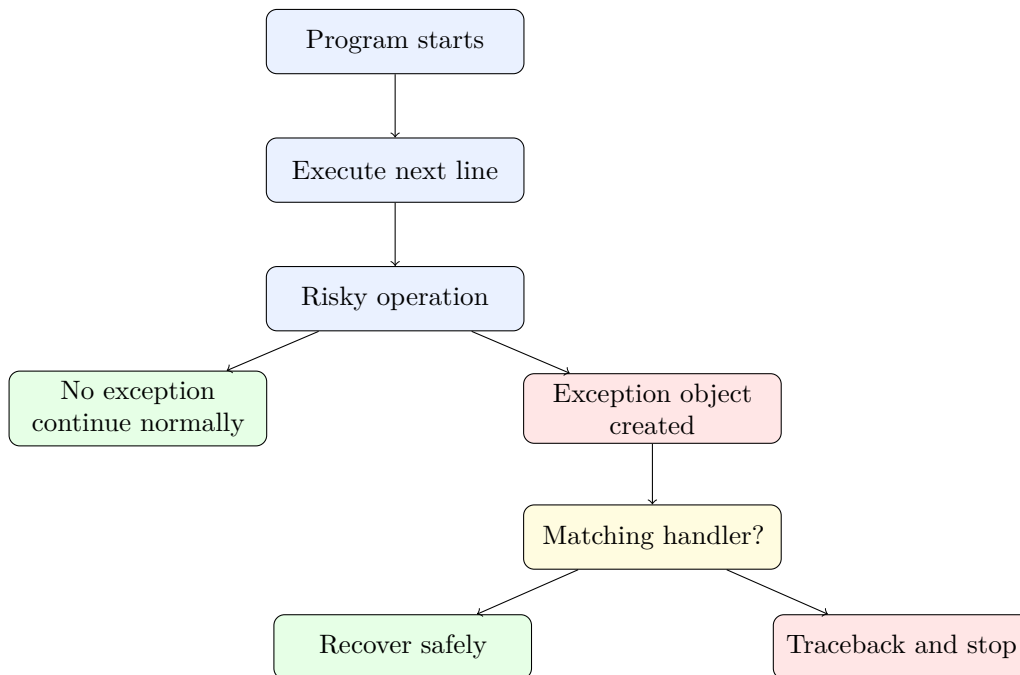
Without exception handling, one unexpected input can stop an entire application. In a playlist app, a user might type `two` instead of `2`, choose a track number that does not exist, or submit an empty value. Python cannot guess the user’s intention, so it raises an exception.

Situation	What Goes Wrong	Likely Exception	Prevention Question
Converting input	<code>int("two")</code> cannot create a number	<code>ValueError</code>	Can this value become an integer?
List lookup	User requests item 5 from a 3-item list	<code>IndexError</code>	Is the index in range?
Dictionary lookup	Program asks for a missing key	<code>KeyError</code>	Does this key exist?
Mixed operations	Code adds text to a number	<code>TypeError</code>	Are the types compatible?
Misspelled name	Code calls <code>prnt()</code>	<code>NameError</code>	Is the name defined and spelled correctly?
Missing file	Program opens a file that is absent	<code>FileNotFoundError</code>	Does the file exist here?

1.3 Formal Explanation

Python normally executes statements from top to bottom. When an exception occurs, Python creates an exception object, stops the current normal flow, and searches for a matching handler. If no handler exists, Python prints a traceback and terminates the program.

1.4 Visual Representation



1.5 Syntax Preview

```
try:
    risky_operation()
except ValueError:
    recover_from_bad_value()
```

- **try:** marks the code where an expected risk may happen.
- **except:** defines what to do if a matching exception occurs.
- **ValueError:** the specific exception type being handled.
- **Indentation:** tells Python which statements belong to each block.

1.6 Step-by-Step Execution

1. Python enters the **try** block.
2. Python runs each line normally.
3. If no exception occurs, the **except** block is skipped.
4. If a matching exception occurs, the rest of the **try** block is skipped.
5. Python jumps to the matching **except** block.
6. After the handler finishes, execution continues after the exception structure.

1.7 Five Examples

Example 1: Very Basic Number Conversion

Problem. Convert text into an integer.

```
text = "42"  
number = int(text)  
print(number + 1)
```

Output.

```
43
```

Walkthrough. Python reads a valid numeric string, converts it to integer 42, then adds 1. No exception occurs.

Example 2: Beginner Invalid Input

Problem. See what happens when text cannot become a number.

```
text = "forty-two"  
number = int(text)  
print(number)
```

Output.

```
ValueError: invalid literal for int() with base 10: 'forty-two'
```

Walkthrough. Python reaches `int(text)`, discovers the value is not numeric, creates a `ValueError`, and stops because there is no handler.

Example 3: Intermediate List Access

Problem. Access a playlist item using an index.

```
playlist = ["Song A", "Song B", "Song C"]  
print(playlist[2])
```

Output.

```
Song C
```

Walkthrough. Lists start at index 0, so index 2 is the third item. This succeeds.

Example 4: Practical Missing Dictionary Key

Problem. Retrieve a saved phone number.

```
contacts = {"Maya": "555-0101"}  
print(contacts["Omar"])
```

Output.

```
KeyError: 'Omar'
```

Walkthrough. The dictionary has no key named "Omar". Python raises `KeyError` because exact-key lookup requires the key to exist.

Example 5: Advanced Failure Prediction

Problem. Predict multiple possible failures before writing handlers.

```
prices = {"notebook": 3.50, "pen": 1.20}
item = input("Item: ").lower()
quantity = int(input("Quantity: "))
print(prices[item] * quantity)
```

Possible Output.

```
ValueError: invalid literal for int() with base 10: 'many'
# or
KeyError: 'eraser'
```

Walkthrough. `int()` can fail if quantity is not numeric. Dictionary lookup can fail if the item is unknown. Professional developers identify these risks before adding handlers.

1.8 Real-World Applications

- **Banking systems:** reject invalid withdrawal amounts before updating balances.
- **ATM software:** handle unreadable cards, wrong PIN attempts, and cash dispenser failures.
- **Hospital software:** prevent missing patient IDs from crashing clinical workflows.
- **Payment gateways:** recover from declined cards, network timeouts, and duplicate transactions.
- **Data pipelines:** skip or quarantine corrupt records while preserving the full batch process.

1.9 Edge Cases

Scenario	What Happens	Why	Fix and Best Practice
Empty input	<code>ValueError</code> during conversion	Empty string is not a number	Check for blank input before conversion.
Negative index	May return an item instead of failing	Python supports negative indexing	Validate user-facing numbers explicitly.
Wrong case key	<code>KeyError</code>	Dictionary keys are exact	Normalize with <code>lower()</code> or use controlled choices.
Missing variable	<code>NameError</code>	Name was never assigned	Define names before use; use editor linting.
Silent failure	Program seems to work but data is wrong	Error was swallowed	Never use empty handlers such as <code>except: pass</code> .

1.10 Common Mistakes

1. **Wrong:** Catch every error with bare `except:`. **Correct:** Catch the specific exception you expect.
2. **Wrong:** Put an entire program inside one `try`. **Correct:** Protect only the risky operation.
3. **Wrong:** Ignore the traceback. **Correct:** Read the final line first, then locate the failing line.
4. **Wrong:** Treat all invalid input the same. **Correct:** Distinguish conversion errors from range errors.
5. **Wrong:** Print an error and continue blindly. **Correct:** stop, retry, return a safe value, or raise a clear exception.

1.11 Workbook Tasks

Complete these without writing full Python solutions in this book.

1. **Easy:** List three inputs that could break a calculator. Inputs: numbers and operators. Expected: identify likely exception names. Constraint: no code. Hint pseudocode: START → read expression parts → inspect conversion risks → inspect operation risks → END.
2. **Beginner:** Predict the exception for `int("7.5")`. Input: string value. Expected: exception name and reason. Hint pseudocode: START → inspect string format → compare with integer format → name failure → END.
3. **Intermediate:** Analyze a playlist with three songs and user choice 0. Expected: explain whether Python fails and why user logic may still be wrong. Hint pseudocode: START → convert choice → subtract one → inspect resulting index → END.
4. **Practical:** Design a contact lookup safety plan. Inputs: contact dictionary and requested name. Expected: behavior for found and missing names. Hint pseudocode: START → normalize name → check existence → display number or friendly message → END.
5. **Advanced:** Build a risk table for an online order form. Inputs: item, quantity, payment token. Expected: five possible failures. Hint pseudocode: START → inspect each input → map each risk to exception or validation failure → define safe response → END.

1.12 Interview Questions

1. What is an exception, and how is it different from a normal return value?
2. Why should a developer predict failures before adding `try/except`?
3. A program raises `IndexError` for user choice 4 in a three-item list. Explain the bug.
4. Design an error strategy for a login form that can receive blank usernames and invalid passwords.
5. When could handling an exception make software less reliable rather than more reliable?

Summary Card

Exceptions are not enemies; they are signals. Professional Python code predicts risky operations, handles expected failures, and lets unexpected programming bugs remain visible during development.

Module 2

Tracebacks: Reading Python's Failure Report

2.1 Introduction

A **traceback** is Python's diagnostic report after an unhandled exception. It tells you what failed, where it failed, and how execution reached that point.

Intuition First

A traceback is a detective's case file. The last line names the crime. The lines above it show the path to the crime scene.

2.2 Formal Explanation

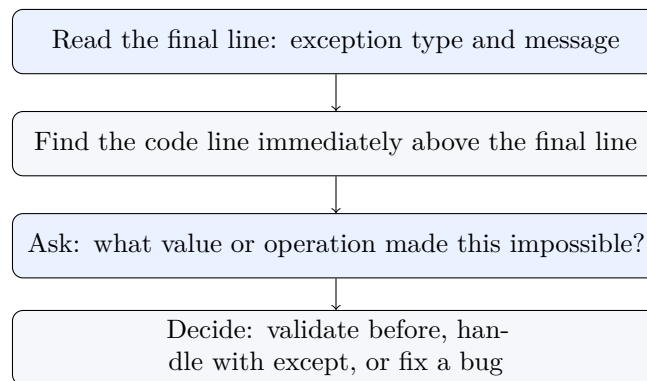
Tracebacks display the call stack from the outermost call toward the failing operation. Beginners often read from the top and feel lost. Professionals usually inspect the final line first because it contains the exception type and message.

2.3 Traceback Anatomy

```
Traceback (most recent call last):
  File "playlist_manager.py", line 8, in <module>
    track_index = int(choice) - 1
ValueError: invalid literal for int() with base 10: 'two'
```

- **Traceback:** Python is reporting the path to failure.
- **File:** the file where a stack frame ran.
- **line 8:** the line Python was executing.
- **Code preview:** the exact statement that failed.
- **ValueError:** the exception class.
- **Message:** the specific reason for the failure.

2.4 Visual Debugging Process



2.5 Five Examples

Example 1: Very Basic ValueError

Problem. Interpret a failed integer conversion.

```
age = int("ten")
```

Output.

```
ValueError: invalid literal for int() with base 10: 'ten'
```

Walkthrough. The final line says `ValueError`. The value `"ten"` has meaning to humans but not to `int()`.

Example 2: Beginner TypeError

Problem. Combine incompatible types.

```
print("Total: " + 50)
```

Output.

```
TypeError: can only concatenate str (not "int") to str
```

Walkthrough. Python can join text to text, not text to integer. Convert the number with `str(50)` or use an f-string.

Example 3: Intermediate IndexError

Problem. Read a missing list position.

```
students = ["Alex", "Blake"]  
print(students[2])
```

Output.

```
IndexError: list index out of range
```

Walkthrough. The list has positions 0 and 1. Position 2 does not exist.

Example 4: Practical KeyError

Problem. Locate a missing profile field.

```
profile = {"name": "Lina", "email": "lina@example.com"}
print(profile["phone"])
```

Output.

```
KeyError: 'phone'
```

Walkthrough. The traceback names the missing key. The fix may be to require the field, use `get()`, or ask the user to complete the profile.

Example 5: Advanced Nested Call Stack

Problem. Understand a failure inside a function.

```
def parse_quantity(text):
    return int(text)

def checkout(raw_quantity):
    quantity = parse_quantity(raw_quantity)
    return quantity * 10

checkout("many")
```

Output.

```
ValueError: invalid literal for int() with base 10: 'many'
```

Walkthrough. The call begins in `checkout`, enters `parse_quantity`, and fails at `int(text)`. The traceback stack helps locate the internal failing function.

2.6 Applications, Edge Cases, and Mistakes

Industry Use

Tracebacks power application logs, error monitoring dashboards, failed job reports, test failure output, and incident investigations.

- **Applications:** web request logs, background job failures, data import reports, automated tests, command-line tools.
- **Edge cases:** long stack traces, repeated wrapper functions, hidden original errors, missing source files, misleading top lines.
- **Mistakes:** reading only the first line, ignoring exception type, changing random code, hiding tracebacks with broad handlers, forgetting the input that caused failure.

2.7 Practice Tasks with Pseudocode

1. **Easy:** Given a one-line `ValueError`, identify type and message. Pseudocode: START → read final line → split at colon → label type and message → END.
2. **Beginner:** Explain a `NameError` caused by `tot1`. Pseudocode: START → inspect failing name → compare intended names → identify spelling issue → END.
3. **Intermediate:** Trace a function call that fails inside a helper. Pseudocode: START → read stack from bottom → locate helper frame → inspect value → END.

4. **Practical:** Create a debugging checklist for a crashed file reader. Pseudocode: START → record file path → read final line → inspect open call → verify file existence → END.
5. **Advanced:** Distinguish an expected user error from a programmer bug in a traceback. Pseudocode: START → classify source of bad value → decide validation or code fix → document action → END.

2.8 Interview Questions

1. Why is the final line of a traceback important?
2. What information does a stack frame provide?
3. How would you debug a traceback that points into a helper function?
4. When should a traceback be shown to developers but hidden from users?
5. Design a logging format that preserves enough traceback information for production debugging.

Module 3

Core Handling: try and except

3.1 Introduction

`try/except` lets your program attempt a risky operation and recover from a specific expected failure.

Warning

Exception handling is not a way to pretend errors do not exist. It is a way to make a deliberate recovery plan.

3.2 Intuition First

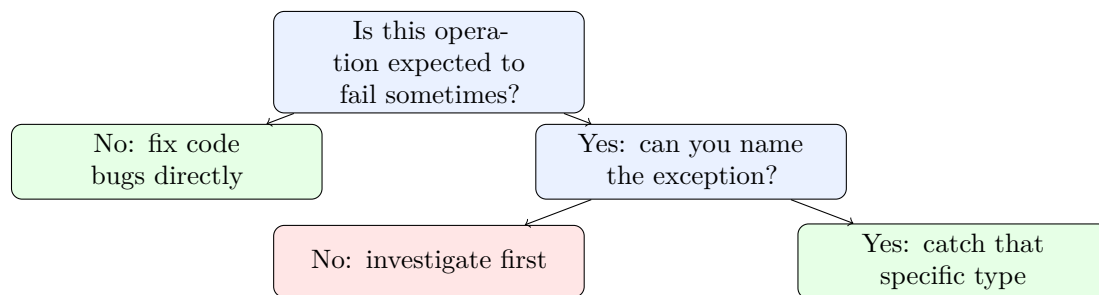
Imagine a cashier scanning an item. Most barcodes work. If one barcode is damaged, the cashier does not close the entire store; they manually enter the code or ask for help. The damaged barcode is the exception. The manual process is the handler.

3.3 Formal Syntax

```
try:
    risky_code
except SpecificException:
    recovery_code
```

Component	Meaning
<code>try</code>	Start a protected block containing code that may fail for an expected reason.
Colon	Introduces an indented block.
Indented code	Statements Python will attempt to run.
<code>except</code>	Starts a handler block.
Exception class	The specific category of problem the handler can catch.
Recovery code	Friendly message, retry, fallback value, or safe exit.

3.4 Decision Tree



3.5 Five Examples

Example 1: Very Basic Conversion Handler

Problem. Handle a non-numeric age.

```

try:
    age = int("unknown")
except ValueError:
    print("Please enter age using digits.")
  
```

Output.

```
Please enter age using digits.
```

Walkthrough. `int()` raises `ValueError`. The matching handler runs and prints a friendly recovery message.

Example 2: Beginner Playlist Manager

Problem. Convert a user's track number safely.

```

playlist = ["Bohemian Rhapsody", "Hotel California", "Stairway to Heaven"]
choice = "two"

try:
    track_index = int(choice) - 1
    print(playlist[track_index])
except ValueError:
    print("Enter a whole number, not words.")
except IndexError:
    print("That track number does not exist.")
  
```

Output.

```
Enter a whole number, not words.
```

Walkthrough. Conversion fails before list access. Python skips the rest of `try` and enters `except ValueError`.

Example 3: Intermediate Separate Risks

Problem. Isolate conversion risk from list-index risk.

```

playlist = ["Song A", "Song B", "Song C"]
choice = "5"

try:
  
```

```

    track_number = int(choice)
except ValueError:
    print("Invalid number format.")
else:
    try:
        print(playlist[track_number - 1])
    except IndexError:
        print("Track number outside playlist range.")

```

Output.

Track number outside playlist **range**.

Walkthrough. Conversion succeeds, so the list lookup is attempted. Index 4 does not exist.

Example 4: Practical File Open

Problem. Read a configuration file only if it exists.

```

try:
    file = open("settings.txt", "r")
    content = file.read()
    file.close()
except FileNotFoundError:
    print("Settings file was not found. Using defaults.")

```

Output.

Settings **file** was **not** found. Using defaults.

Walkthrough. If the file is absent, `open()` raises `FileNotFoundError`. The fallback is to use default settings.

Example 5: Advanced Capturing the Exception Object

Problem. Show the exact reason while still controlling the message.

```

try:
    result = 100 / 0
except ZeroDivisionError as error:
    print(f"Calculation failed: {error}")

```

Output.

Calculation failed: division by zero

Walkthrough. `as error` stores the exception object, allowing the handler to include the original diagnostic message.

3.6 Comparison Table

Approach	Use Case	Risk
Bare <code>except</code> :	Rare top-level emergency shut-down	Hides bugs and system-exit signals.
Specific <code>except ValueError</code>	Expected invalid user value	Requires knowing the likely failure.
Multiple handlers	Different recovery actions	Order matters for related exception classes.
Validation before operation	Prevent predictable invalid states	Can duplicate checks if overused.

3.7 Real-World Applications

1. Login forms catch invalid input formats without exposing server errors.
2. Payment processors catch declined-card and timeout failures separately.
3. File import tools catch missing files and malformed rows.
4. Sensor systems catch unreadable measurements and continue polling.
5. Web APIs catch bad request data and return controlled error responses.

3.8 Edge Cases

1. **Broad handler hides typos.** Scenario: `print()` inside `try`. It may be caught by a broad handler. Fix: catch specific exceptions.
2. **Too much code in `try`.** Scenario: handler catches a later failure you did not expect. Fix: keep risky block small.
3. **Handler uses undefined variable.** Scenario: recovery code references a value never assigned. Fix: initialize safe defaults.
4. **Multiple failures share one type.** Scenario: two `int()` calls both raise `ValueError`. Fix: separate steps or improve messages.
5. **User-facing index starts at one.** Scenario: users type 1 but lists start at 0. Fix: convert carefully and validate range.

3.9 Common Mistakes

1. Wrong: `except: print("bad")`. Correct: `except ValueError: print("Enter digits")`. Specific handling preserves debugging.
2. Wrong: handling programmer typos as user errors. Correct: fix the typo and let development tools detect it.
3. Wrong: continuing with invalid state. Correct: return, retry, or stop the workflow safely.
4. Wrong: showing raw tracebacks to end users. Correct: log details for developers and show friendly messages to users.
5. Wrong: using exceptions for ordinary branching. Correct: use `if` when the condition is simple and expected.

3.10 Practice Tasks with Pseudocode

1. **Easy:** Handle `int()` conversion for a single age. Pseudocode: START → read text → TRY convert → IF conversion fails show message → END.
2. **Beginner:** Handle missing dictionary contact. Pseudocode: START → read name → TRY lookup → EXCEPT missing key show not found → END.
3. **Intermediate:** Handle two playlist failures separately. Pseudocode: START → TRY convert choice → EXCEPT bad number → TRY access list → EXCEPT bad index → END.
4. **Practical:** Read a file or use default text. Pseudocode: START → TRY open file → read contents → EXCEPT file missing set default → END.

5. **Advanced:** Capture an exception object for logging. Pseudocode: START → TRY risky calculation → EXCEPT named error AS object → log message → show safe output → END.

3.11 Interview Questions

1. What is the purpose of `try/except`?
2. Why is a bare `except` usually discouraged?
3. How do you choose which exception type to catch?
4. Debug this design: one giant `try` block around an entire program.
5. Design exception handling for a checkout workflow with inventory and payment risks.

Module 4

Clean Control Flow: `else` and `finally`

4.1 Introduction

`else` and `finally` make exception-handling code clearer. `else` means “run this only if the `try` block succeeded.” `finally` means “run this no matter what.”

Tip

Use `else` to separate risky code from success-only code. Use `finally` to clean up resources such as files, connections, locks, or temporary states.

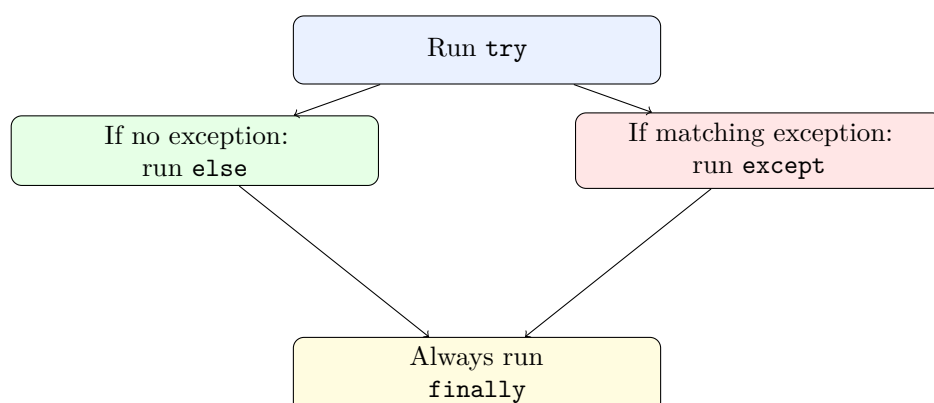
4.2 Intuition First

`else` is the “all clear” announcement after a safety inspection. `finally` is like turning off the lights and locking the door whether the event succeeded, failed, or was cancelled.

4.3 Formal Syntax

```
try:
    risky_code()
except ValueError:
    handle_bad_value()
else:
    run_only_after_success()
finally:
    cleanup_always()
```

4.4 Execution Flow



4.5 Five Examples

Example 1: Very Basic else

Problem. Print a success message only after conversion succeeds.

```
try:
    number = int("8")
except ValueError:
    print("Bad number")
else:
    print(number * 2)
```

Output.

```
16
```

Walkthrough. No exception occurs, so Python skips `except` and runs `else`.

Example 2: Beginner Dictionary Recommendation

Problem. Recommend a movie for a known genre.

```
recommendations = {"sci-fi": "Interstellar", "comedy": "Paddington 2"}
genre = "sci-fi"

try:
    movie = recommendations[genre]
except KeyError:
    print("No recommendation for that genre.")
else:
    print(f"Tonight you could watch {movie}.")
```

Output.

```
Tonight you could watch Interstellar.
```

Walkthrough. The lookup succeeds. The success-only display belongs in `else`, not inside the risky lookup block.

Example 3: Intermediate finally

Problem. Always show that a process ended.

```
try:
    value = int("oops")
except ValueError:
    print("Could not parse value.")
finally:
    print("Parsing attempt finished.")
```

Output.

```
Could not parse value.
Parsing attempt finished.
```

Walkthrough. The handler runs because conversion fails. Then `finally` runs regardless.

Example 4: Practical File Cleanup

Problem. Close a file after reading.

```
file = None
try:
    file = open("data.txt", "r")
    text = file.read()
except FileNotFoundError:
    print("File missing.")
else:
    print("Read completed.")
finally:
    if file is not None:
        file.close()
```

Output.

```
File missing.
# or, if the file exists:
Read completed.
```

Walkthrough. `finally` protects cleanup. The guard prevents calling `close()` on `None`.

Example 5: Advanced Transaction-Like Flow

Problem. Separate validation, success action, and cleanup.

```
locked = False
try:
    locked = True
    quantity = int("3")
except ValueError:
    print("Invalid quantity.")
else:
    print(f"Reserved {quantity} items.")
finally:
    if locked:
        print("Inventory lock released.")
```

Output.

```
Reserved 3 items.
Inventory lock released.
```

Walkthrough. The success action happens only after valid conversion. Cleanup happens even if future code fails.

4.6 Real-World Applications

- Database sessions commit on success and close connections in cleanup.
- File processors read only if opening succeeds and close handles afterward.
- Web servers log request completion whether the request succeeds or fails.
- Payment workflows release inventory holds after success or failure.
- IoT programs reset device flags after sensor-read attempts.

4.7 Edge Cases

1. **Exception in else.** It is not caught by the earlier **except**. Fix: keep **else** success code simple or add a new handler where needed.
2. **Exception in finally.** It can hide the original exception. Fix: avoid risky cleanup code.
3. **Return in finally.** It can override earlier returns. Fix: do not return from **finally** unless deliberately designed.
4. **Unassigned resource.** Cleanup references a variable never created. Fix: initialize to **None** before **try**.
5. **Overusing else.** Tiny scripts may become noisy. Fix: use it when it improves clarity.

4.8 Common Mistakes

1. Wrong: put success messages inside **try**. Correct: put non-risky success actions in **else**.
2. Wrong: assume **finally** means success. Correct: **finally** means always.
3. Wrong: close resources only in **else**. Correct: close in **finally** or use **with**.
4. Wrong: write complex business logic in **finally**. Correct: keep cleanup predictable.
5. Wrong: forget that **else** is skipped after exceptions. Correct: use it only for success-only behavior.

4.9 Practice Tasks with Pseudocode

1. **Easy:** Add a success-only message after parsing a number. Pseudocode: START → TRY parse → EXCEPT show error → ELSE show success → END.
2. **Beginner:** Use **finally** to print “done” after any attempt. Pseudocode: START → TRY risky work → EXCEPT recover → FINALLY print done → END.
3. **Intermediate:** Clean up a pretend connection flag. Pseudocode: START → set connected false → TRY connect → ELSE use service → FINALLY disconnect if connected → END.
4. **Practical:** Design file-read flow with missing-file fallback. Pseudocode: START → initialize file → TRY open/read → EXCEPT default content → FINALLY close if opened → END.
5. **Advanced:** Create a transaction map with validate, reserve, confirm, cleanup. Pseudocode: START → TRY validate → EXCEPT reject → ELSE confirm → FINALLY release temporary hold → END.

4.10 Interview Questions

1. When does **else** run in exception handling?
2. Why might **else** be cleaner than putting everything in **try**?
3. Give a real case where **finally** is essential.
4. What danger exists if **finally** itself raises an exception?
5. Design a safe file workflow using **try**, **except**, **else**, and **finally**.

Module 5

Raising Exceptions Intentionally

5.1 Introduction

Python raises built-in exceptions when Python's own rules are broken. You can also use `raise` when your program's business rules are broken.

Warning

A printed error message is not the same as an exception. `print("Error")` informs a human; `raise` changes program control flow.

5.2 Intuition First

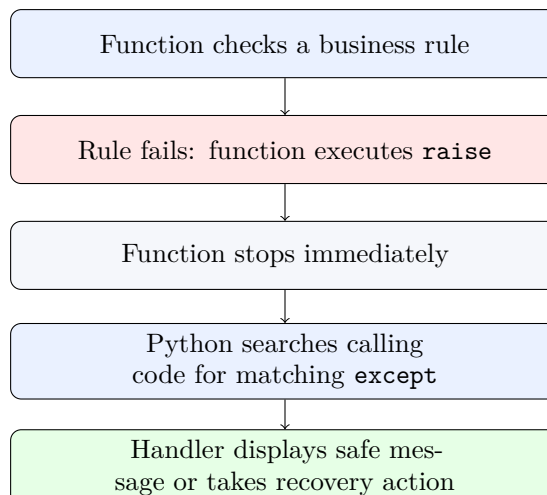
`raise` is an emergency brake. If a ticketing system allows at most five tickets, Python does not know that rule automatically. Your code must stop the transaction when the rule is violated.

5.3 Formal Syntax

```
raise ValueError("Quantity cannot exceed 5")
```

- `raise`: intentionally trigger an exception.
- `ValueError`: exception type that describes an inappropriate value.
- Message string: human-readable explanation attached to the exception.

5.4 Raise-and-Catch Lifecycle



5.5 Five Examples

Example 1: Very Basic Manual Error

Problem. Stop when a number is negative.

```
number = -3
if number < 0:
    raise ValueError("Number must not be negative.")
```

Output.

```
ValueError: Number must not be negative.
```

Walkthrough. Python did not naturally object to -3. Your rule did, so your code raised `ValueError`.

Example 2: Beginner Ticket Limit

Problem. Enforce a maximum of five tickets.

```
def buy_tickets(quantity):
    if quantity > 5:
        raise ValueError("Cannot purchase more than 5 tickets.")
    print(f"Purchased {quantity} tickets.")

buy_tickets(12)
```

Output.

```
ValueError: Cannot purchase more than 5 tickets.
```

Walkthrough. The function rejects a valid Python integer because it violates the application rule.

Example 3: Intermediate Raise and Catch

Problem. Validate an alarm hour.

```
def set_alarm_hour(hour):
    if hour < 0 or hour > 23:
        raise ValueError("Clock hours must be between 0 and 23.")
    print(f"Alarm set for {hour}:00.")

try:
    requested_hour = int("25")
    set_alarm_hour(requested_hour)
except ValueError as error:
    print(f"Setup failed: {error}")
```

Output.

```
Setup failed: Clock hours must be between 0 and 23.
```

Walkthrough. `int("25")` succeeds. The custom rule inside `set_alarm_hour` fails and raises a `ValueError` caught by the outer handler.

Example 4: Practical Order Validation

Problem. Reject impossible order quantities.

```
def validate_order(quantity, stock):
    if quantity <= 0:
        raise ValueError("Quantity must be positive.")
    if quantity > stock:
```

```
        raise ValueError("Quantity exceeds available stock.")
    return True

try:
    validate_order(7, stock=3)
except ValueError as error:
    print(f"Order rejected: {error}")
```

Output.

```
Order rejected: Quantity exceeds available stock.
```

Walkthrough. Validation functions often raise exceptions because they are gatekeepers for unsafe states.

Example 5: Advanced Re-Raising

Problem. Log an error locally, then let another layer handle it.

```
def download_file():
    try:
        raise OSError("Network connection lost mid-download.")
    except OSError as error:
        print(f"[LOG] Download failed: {error}")
        raise

try:
    download_file()
except OSError:
    print("User message: Please try again later.")
```

Output.

```
[LOG] Download failed: Network connection lost mid-download.
User message: Please try again later.
```

Walkthrough. Bare `raise` inside an `except` block rethrows the same exception with its traceback context.

5.6 Real-World Applications

- Banking code raises when a withdrawal exceeds available balance.
- Ticketing systems raise when purchase limits are exceeded.
- API clients raise when required authentication tokens are missing.
- Data validators raise when records violate schema rules.
- Security systems raise when access attempts violate policy.

5.7 Edge Cases

1. **Raising wrong type.** Scenario: invalid quantity raises `TypeError`. Fix: use `ValueError` when type is right but value is unacceptable.
2. **No message.** Scenario: `raise ValueError`. Fix: attach a clear message.
3. **Raising too late.** Scenario: update database before validation. Fix: validate before side effects.

4. **Bare raise outside handler.** Scenario: used where no active exception exists. Fix: use bare `raise` only inside `except`.
5. **Replacing original error carelessly.** Scenario: handler raises a new vague error. Fix: preserve context or re-raise.

5.8 Common Mistakes

1. Wrong: `print("Too many tickets")` then continue. Correct: `raise ValueError(...)` to stop unsafe flow.
2. Wrong: raise for normal menu choices. Correct: use `if/elif` for ordinary branching.
3. Wrong: raise vague messages. Correct: state the violated rule.
4. Wrong: catch immediately just to hide the error. Correct: catch where meaningful recovery exists.
5. Wrong: re-raise with `raise error` unnecessarily. Correct: use bare `raise` to preserve traceback.

5.9 Practice Tasks with Pseudocode

1. **Easy:** Reject negative ages. Pseudocode: START → read age → IF age less than zero RAISE invalid value → END.
2. **Beginner:** Enforce ticket purchase limit. Pseudocode: START → read quantity → IF quantity greater than limit RAISE rule violation → ELSE approve → END.
3. **Intermediate:** Validate alarm hour after parsing. Pseudocode: START → TRY parse hour → CALL validator → IF outside range RAISE → EXCEPT show setup failure → END.
4. **Practical:** Validate order stock before checkout. Pseudocode: START → compare requested quantity with stock → RAISE if invalid → reserve if valid → END.
5. **Advanced:** Log and re-raise a network error. Pseudocode: START → TRY download → EXCEPT network error log details → RAISE same error → outer handler shows user message → END.

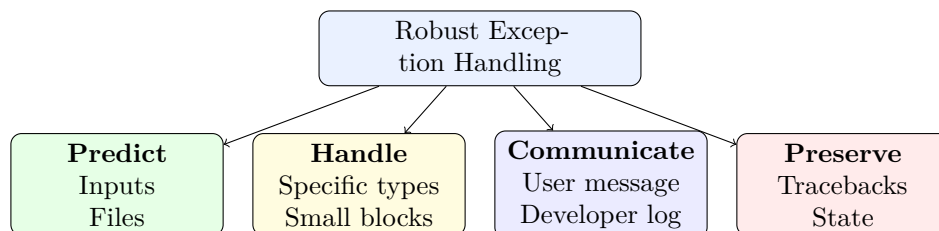
5.10 Interview Questions

1. Why would a programmer intentionally raise an exception?
2. What is the difference between printing an error and raising an error?
3. Which built-in exception would you raise for an invalid value?
4. Explain re-raising and why it is useful for logging.
5. Design validation for a banking withdrawal function using exceptions.

Module 6

Professional Patterns and Production Thinking

6.1 Best-Practice Mind Map



6.2 Professional Design Principles

1. **Catch expected failures only.** Expected user mistakes belong in handlers; programmer mistakes should be fixed.
2. **Keep protected blocks small.** The smaller the `try`, the easier it is to know what failed.
3. **Name the enemy.** Catch `ValueError`, `KeyError`, or `FileNotFoundError`, not everything.
4. **Separate user messages from developer diagnostics.** Users need guidance; developers need details.
5. **Preserve original context.** Re-raise when a higher layer should decide the final recovery.

6.3 Use Cases vs Anti-Patterns

Best Practice	Bad Practice
Validate business rules before side effects.	Update state, then discover invalid input.
Catch <code>FileNotFoundError</code> for optional files.	Catch every exception and assume the file is missing.
Use <code>else</code> for success-only code.	Put long success workflows inside <code>try</code> .
Use <code>finally</code> or <code>with</code> for cleanup.	Leave resources open after failure.
Log technical details and show friendly messages.	Show raw traceback to users.

6.4 Challenge Section

Mini Project 1: Safe Playlist Manager

Build a command-line playlist selector. It must handle non-numeric input, out-of-range choices, and empty input. Add friendly messages and keep conversion risk separate from indexing risk.

Mini Project 2: Event Ticket Checker

Create a ticket-purchase validator. It must raise an exception for quantities below 1 or above 5. The main program must catch the exception and show a user-safe message.

Mini Project 3: Movie Recommendation Tracker

Use a dictionary of genres to recommendations. Handle missing genres, use `else` for successful recommendations, and produce a clear fallback message.

Debugging Exercise

```
try:
    prices = {"pen": 2}
    item = "pencil"
    total = prices[item] * int("three")
except:
    print("Something went wrong")
```

Task. Identify every risk hidden by the broad handler. Rewrite the design in pseudocode with specific handlers.

Code Tracing Exercise

Trace this without running it:

```
def validate(x):
    if x == 0:
        raise ValueError("zero not allowed")
    return 10 / x

try:
    print(validate(0))
except ZeroDivisionError:
    print("division problem")
except ValueError as error:
    print(error)
finally:
    print("done")
```

Predict the output and explain why `ZeroDivisionError` does not run.

Optimization Exercise

Refactor a giant `try` block into smaller units: input parsing, validation, lookup, calculation, and display. Your goal is clarity, not fewer lines.

6.5 Capstone Practice Tasks with Pseudocode

1. **Easy:** Create a failure-prediction table for a calculator. Pseudocode: START → list operations → identify invalid values → map to exception types → END.

2. **Beginner:** Design safe profile lookup. Pseudocode: START → read requested field → TRY dictionary lookup → EXCEPT missing field show guidance → END.
3. **Intermediate:** Combine `try`, `except`, and `else`. Pseudocode: START → TRY risky parse → EXCEPT recover → ELSE process valid value → END.
4. **Practical:** Create a resilient data-import workflow. Pseudocode: START → open file → FOR each row TRY parse → EXCEPT quarantine row → FINALLY close report → END.
5. **Advanced:** Design layered exception handling for a payment service. Pseudocode: START → validate request → RAISE rule errors → TRY payment provider → EXCEPT timeout log and re-raise → top layer returns safe response → END.

6.6 Professional Interview Set

1. Explain exception handling to a beginner using an analogy.
2. What are the risks of `except: pass`?
3. How do `else` and `finally` improve maintainability?
4. How would you handle invalid rows in a million-row data pipeline?
5. Design a production error-handling strategy for checkout: validation, inventory, payment, logging, and user messaging.

Module 7

Cheat Sheets, Revision Notes, Glossary, and Index

7.1 Quick Reference Guide

Pattern	Template
Specific handler	<code>try: ... except ValueError: ...</code>
Capture message	<code>except ValueError as error: print(error)</code>
Success-only code	<code>try: ... except ...: ... else: ...</code>
Always cleanup	<code>try: ... finally: cleanup()</code>
Manual rule failure	<code>raise ValueError("clear message")</code>
Re-raise same error	<code>except OSError: log(); raise</code>

7.2 Exception Selection Card

Revision Card: Which Exception Should I Use?

- **ValueError**: right type, unacceptable value.
- **TypeError**: operation receives incompatible type.
- **IndexError**: sequence index does not exist.
- **KeyError**: dictionary key does not exist.
- **FileNotFoundError**: requested file path cannot be found.
- **OSError**: operating-system or input/output problem.

7.3 Chapter Summaries

- **Foundations**: exceptions interrupt normal flow and require safe recovery paths.
- **Tracebacks**: read the final line first, then inspect the failing frame.
- **try/except**: protect small risky operations and catch specific exceptions.
- **else/finally**: separate success-only work from cleanup that must always happen.
- **raise**: enforce your own program rules and preserve context when re-raising.
- **Professional patterns**: design for clear user messages, useful logs, and safe state.

7.4 Final Revision Questions

1. What problem does exception handling solve?
2. Why should you avoid broad handlers?
3. When does `else` run?
4. Why is `finally` useful for cleanup?
5. Why is `raise` stronger than `print("Error")`?
6. How can a handler accidentally hide a programming bug?
7. What should users see when an exception occurs in production?
8. What should developers see in logs?

7.5 Glossary

Exception	An object representing an abnormal event that interrupts normal execution.
Traceback	Python's report showing where an unhandled exception occurred.
Handler	Code inside an <code>except</code> block that responds to a matching exception.
try	A block where risky code is attempted.
except	A block that catches and responds to a specific exception type.
else	A block that runs only when the <code>try</code> block succeeds.
finally	A block that runs whether success or failure occurs.
raise	A statement that intentionally triggers an exception.
Re-raise	Raising the current exception again, usually after logging or adding context.
Validation	Checking that data satisfies rules before the program relies on it.

7.6 Further Learning Recommendations

- Practice reading tracebacks from real failed scripts.
- Learn context managers and the `with` statement for resource cleanup.
- Study custom exception classes after mastering built-in exceptions.
- Explore logging with Python's `logging` module.
- Apply exception strategy in small projects: calculators, file readers, API clients, and data importers.

Index

else block, [15](#)

except, [10](#)

exception, [1](#)

finally block, [15](#)

raise, [19](#)

traceback, [6](#)

try, [10](#)